Research Seminar

Group 4

10. DEEP ORDER FLOW IMBALANCE: EXTRACTING ALPHA AT MULTIPLE HORIZONS FROM THE LIMIT ORDER BOOK

PETTER N. KOLM, JEREMY TURIEL AND NICHOLAS WESTRAY

Maselskas Artemiy 241
Polishchuk Anastasia 241
Sudarikova Maria 241

Why is High-Frequency Forecasting so hard?

- The Limit Order Book (LOB): A massive, constantly changing list of all buy and sell orders of equities.
- The Challenge: Raw LOB data is non-stationary and incredibly noisy.
- Traditional approach: Feed raw prices and volumes into a model (rarely works well).

2.1.1. *Order Book States.* Throughout the rest of the article, we consider the first ten levels of the order book for each stock. We define the *state of the order book at time* t as the vector of price and volume information for the top ten (non-empty) bid and ask levels

$$\mathbf{s}_t^{\text{LOB}} := (a_t^1, v_t^{1,a}, b_t^1, v_t^{1,b}, \ldots, a_t^{10}, v_t^{10,a}, b_t^{10}, v_t^{10,b})^\top \in \mathbb{R}^{40}, \tag{1}$$

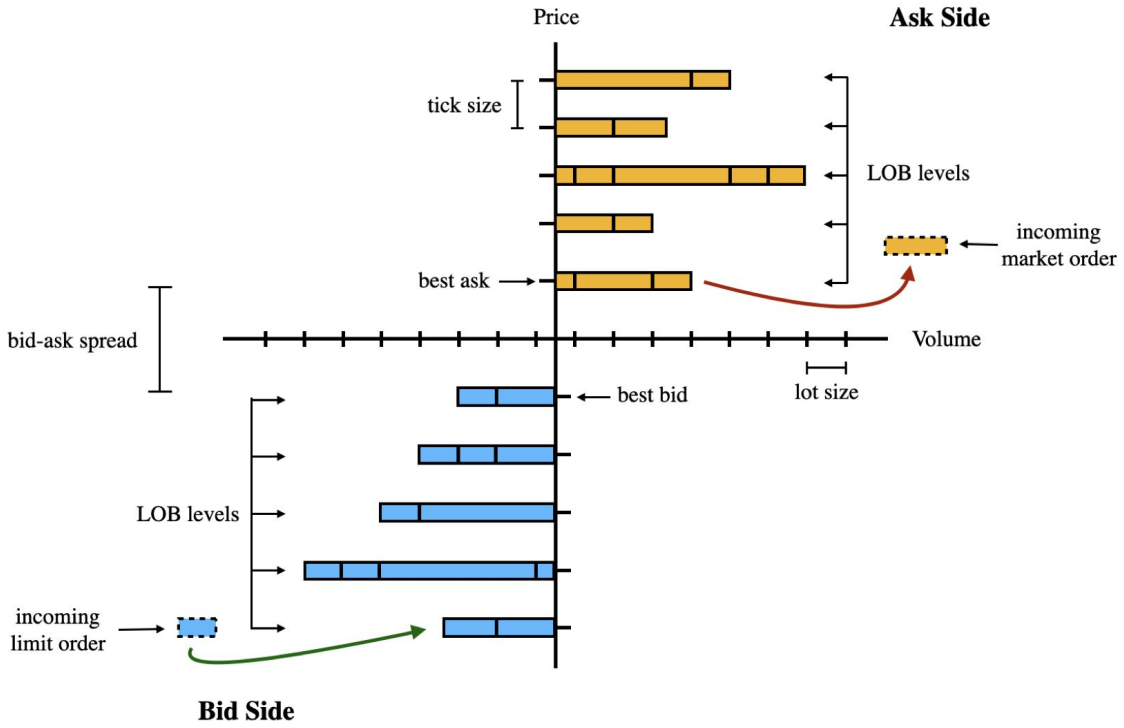


FIGURE 1. An illustration of a limit order book. We observe a market order being filled at the ask side and an incoming limit order being added to the bid side.

The Paper's Insight: Order Flow

- Stop looking at the snapshot. Look at the flow.
- Order Flow (OF): The change in volume at each price level.
- Order Flow Imbalance (OFI): The difference between buy pressure and sell pressure (bOF - aOF).
- Why it works: OF and OFI transform non-stationary prices into a stationary time series.
- Result: Models trained on OF significantly outperform models trained on raw LOB.

Instead of looking at a static picture of the book, we look at the flow - how orders are arriving and canceling. This is called Order Flow, or OF. The original paper proved that this stationary transformation is the secret sauce for high-frequency predictions.

2.1.2. *Bid-Ask Order Flow and Order Flow Imbalance.* For each stock and date, we let $t \in \{1, \dots, T\}$ denote an enumeration of all order book updates on that day. Given two consecutive order book states for the stock at $t-1$ and t of the form (1), we define the *bid order flows* (bOF) and *ask order flows* (aOF) at time t as the vectors $\mathbf{bOF}_t, \mathbf{aOF}_t \in \mathbb{R}^{10}$, where each component is given by

$$\mathrm{bOF}_{t,i} := \begin{cases} v_t^{i,b}\,, & \text{if } b_t^i > b_{t-1}^i\,, \\ v_t^{i,b} - v_{t-1}^{i,b}\,, & \text{if } b_t^i = b_{t-1}^i\,, \\ -v_t^{i,b}\,, & \text{if } b_t^i < b_{t-1}^i\,, \end{cases} \tag{2}$$

$$\mathrm{aOF}_{t,i} := \begin{cases} -v_t^{i,a}\,, & \text{if } a_t^i > a_{t-1}^i\,, \\ v_t^{i,a} - v_{t-1}^{i,a}\,, & \text{if } a_t^i = a_{t-1}^i\,, \\ v_t^{i,a}\,, & \text{if } a_t^i < a_{t-1}^i\,, \end{cases} \tag{3}$$

for $i = 1, \dots, 10$. By concatenating and subtracting the bid and ask order flows at time t, we obtain the *order flow* (OF)

$$\mathbf{OF}_t := \begin{pmatrix} \mathbf{bOF}_t \\ \mathbf{aOF}_t \end{pmatrix} \in \mathbb{R}^{20}\,, \tag{4}$$

and the *order flow imbalance* (OFI)

$$\mathbf{OFI}_t := \mathbf{bOF}_t - \mathbf{aOF}_t \in \mathbb{R}^{10}\,, \tag{5}$$

What are we forecasting? Alpha term structure

What are we actually predicting? We don't just guess one price. We predict a whole curve - an Alpha Term Structure. The variable is the future mid-price return, measured in dollars, minus a 10-millisecond latency. We predict this return for 10 different horizons, ranging from a fraction of a price change to two full price changes. This tells us not just *where* the price will go, but *when* it will get there.

3.2. **Methodology.** We focus on simultaneously forecasting future mid-price returns across multiple horizons. In particular, for each stock and time t, our multi-horizon forecasts are represented as the vector

$$\mathbf{r}_t := (r_{t,1}, \ldots, r_{t,H})^\top \in \mathbb{R}^H\,, \quad \text{where } H \geq 1\,, \tag{14}$$

where H denotes the number of horizons. We refer to (14) as an *alpha term structure* at time t. Our forecasting models are trained at the single stock level and take the form

$$\mathbf{r}_t = \mathbf{g}(\mathbf{x}_t, \mathbf{x}_{t-1}, \ldots, \mathbf{x}_{t-W}) + \boldsymbol{\varepsilon}_t\,, \tag{15}$$

equation (14) stock specific. Specifically, we define a stock specific time increment

$$\Delta t := \frac{2.34 \cdot 10^7}{N}\,, \tag{16}$$

where the numerator is the number of milliseconds in a trading day, and the denominator, N, denotes the average number of non-zero tick-by-tick mid-price returns. This ratio measures how long on average we have to wait for a non-zero mid-price return for a given stock.

We choose an alpha term structure (14) of dimension $H = 10$ and define our forecast horizons, $\{h_k\}_{k=1}^{10}$, as the following multiples of Δt

$$h_k := \frac{1}{5} k \Delta t\,,\ k = 1, \ldots, 10\,. \tag{17}$$

As dependent variables in our single stock forecasting models (15) we use the h_k-horizon mid-price returns $r_{t,k} := p_{t+h_k} - p_{t+\delta\tau}$ (in dollars), where p_t denotes the mid-price at time t and $\delta\tau$ represents a ten millisecond latency buffer. Analo-

Deviations from the Original Study

- **Data access.** IEX DEEP replaces Nasdaq/LOBSTER because the original data were unavailable to us. Different trading venues limit direct comparison of the results.
- **Smaller sample.** 11 stocks, 12 weeks and 3 rolling windows, versus 115 stocks, 48 weeks and 18 windows. Neural models use 6 stocks. Selection favors large-tick stocks and reduces statistical power.
- **Training budget.** Each epoch samples 400,000 training windows. Early stopping uses 200,000 fixed validation windows. Testing covers the full test week. Sampling can affect model fitting and selection.
- **Learning settings.** MLP and LSTM variants use a learning rate of 10^{-4} instead of 10^{-5}. LSTM (3) has a 12-epoch cap. These choices fit local hardware limits, but may affect convergence.
- **ARX estimation.** Closed-form ridge regression with a validation-selected penalty replaces Adam. Additional Adam runs check how this choice affects the comparison.
- **Experiment scope.** OF is the main input. OFI checks cover ARX, LSTM and MLP. Only ARX (Adam) and MLP run on LOB.

Data and Our Replication Setup

- Original Data: LOBSTER, 115 Nasdaq stocks, 48 weeks.
- Our Data: IEX DEEP (Level 2), 11 stocks, 12 weeks.
- Stock Universe (Grouped by Spread):
    i. Large-tick (highly liquid): BAC, VZ, NKE
    ii. Medium-tick: USB, OXY, MO, NEE
    iii. Small-tick (volatile): UBER, DIS, CTSH, BSX
- The Goal: Can we extract alpha from a smaller exchange?

We couldn't get the 10-terabyte LOBSTER dataset. So we used IEX DEEP data. IEX is a smaller exchange, so we scaled down to 11 tickers. We grouped them into small, medium, and large-tick stocks, because as we will see later, the tick size changes everything.

The Architectures Tested

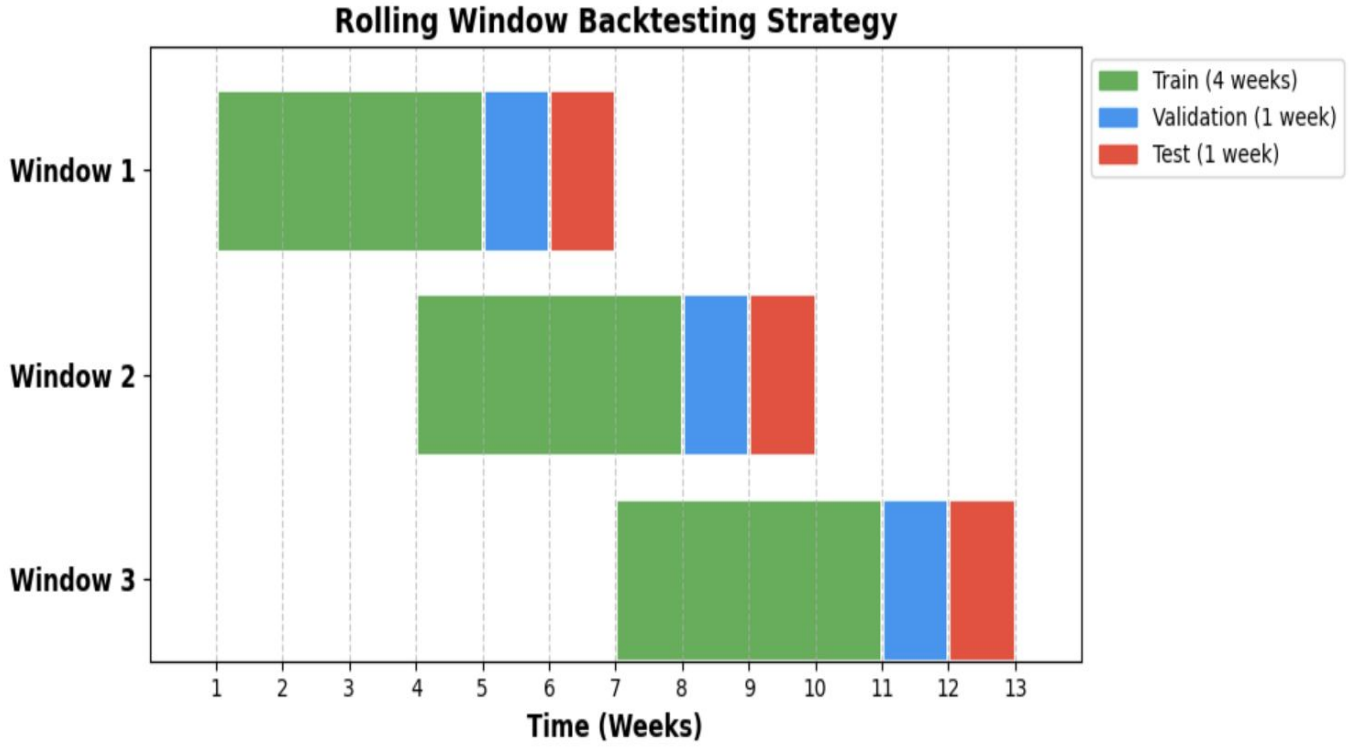
Each model maps a 100-event OF or OFI window to ten forecasts.
The table describes its architecture and the form of the forecasting function.

Model	Architecture	Key feature
ARX	100-event window. Ridge regression with a penalty selected on validation data.	Linear model.
MLP	Dense layers: 512, 512 and 128 units. ReLU activations and a linear output.	Processes all 100 updates at once through nonlinear, fully connected layers.
LSTM	One LSTM layer with 150 units, followed by a linear output layer.	Reads events in order and uses gates to keep or forget information from earlier events.
LSTM (3)	Three stacked LSTM layers with 150 units each, followed by a linear output.	Stacks three LSTM layers, so higher layers process information passed by lower layers.
LSTM–MLP	LSTM with 128 units, a dense layer with 60 ReLU units, and a linear output.	Uses LSTM to summarize the sequence, then an extra nonlinear layer to form forecasts.
CNN–LSTM	Convolution and Inception blocks, an LSTM with 64 units, and a linear output.	CNN combines features across levels and nearby events, then LSTM processes the sequence.

Model Training

How the Pipeline Works

- Rolling Windows: 4 weeks Train -> 1 week Validate -> 1 week Test. Shifted forward by 3 weeks.
- Normalization: Z-score scaling and winsorization (0.5% and 99.5%) based only on training data.
- The CLI: A fully automated Python package (deepofi) to train, evaluate, and compare models.



What Did the Original Authors Find?

- Models trained on Order Flow (OF) drastically beat models trained on raw LOB.
- LSTMs and CNN-LSTMs are the undisputed champions.
- The Decay: Predictive power (R^2(os)) peaks around 1.0 to 2.0 average price changes.
- The Scale: On Nasdaq, the best models achieved an average (R^2(os)) of up to 1.2%.

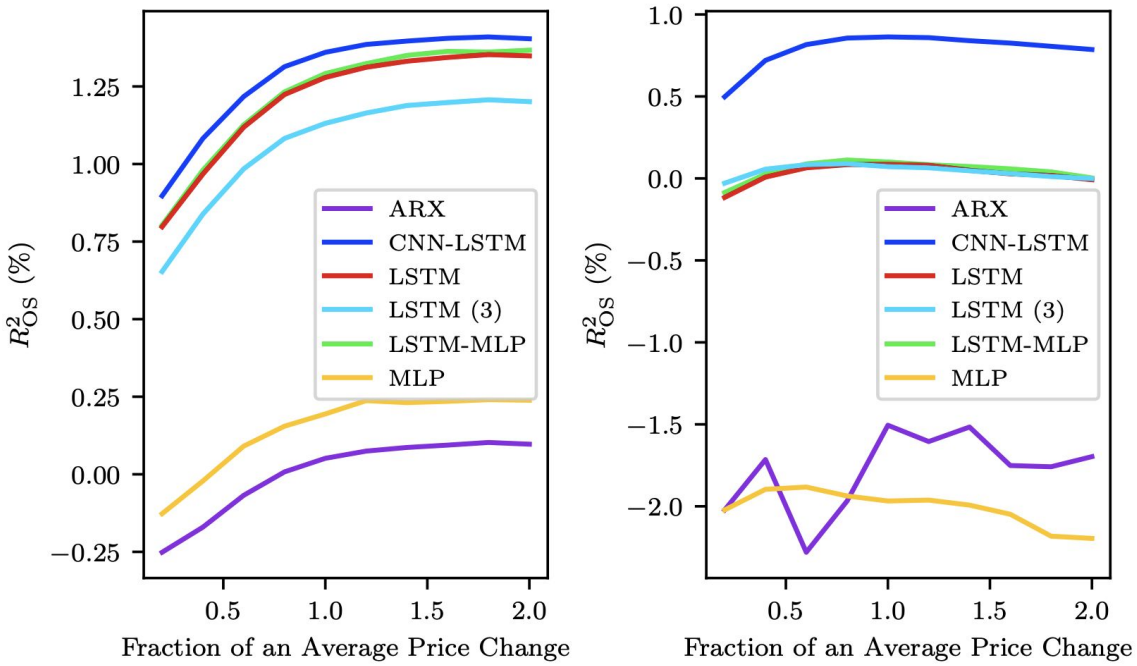
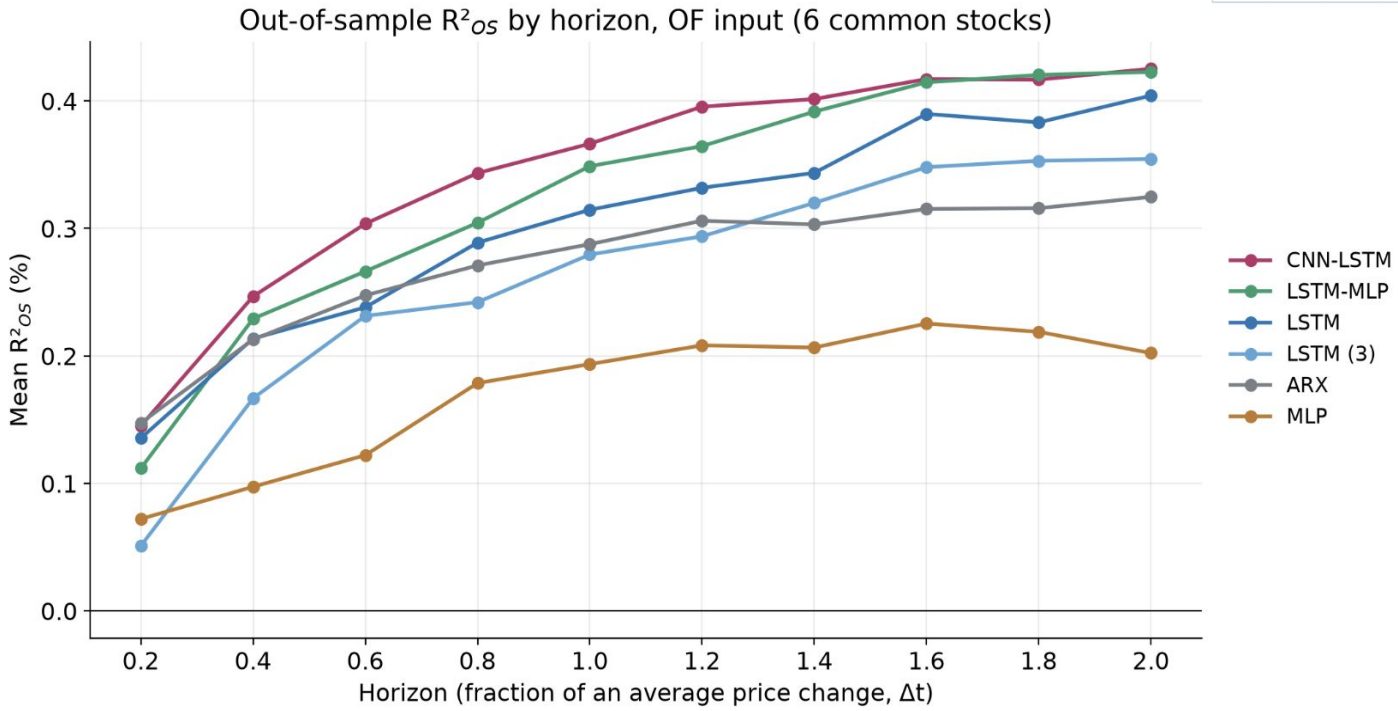


FIGURE 2. Out-of-sample forecasting performance of the models at different horizons from 10 levels of order flows (left panel) and order book states (right panel). Model performance is measured as the average R^2_{OS} across days and stocks. Horizons are given as the fraction of an average price change for each stock. Each model is trained using a rolling-window out-of-sample methodology across 11 months of data.

Replicating the Paper on IEX DEEP (6 Stocks, 3 Windows)

Model	Mean R^2_{OS}	Horizon 0.2Δt	1Δt	2Δt	Stocks with R^2_{OS} significantly > 0 (1%)
CNN-LSTM	**0.35%**	0.15%	0.37%	0.43%	5 of 6
LSTM-MLP	0.33%	0.11%	0.35%	0.42%	4
LSTM	0.30%	0.14%	0.32%	0.40%	3
ARX (ridge)	0.27%	0.15%	0.29%	0.33%	3
LSTM (3)	0.26%	0.05%	0.28%	0.35%	3
MLP	0.17%	0.07%	0.19%	0.20%	3
ARX + CNN-LSTM ensemble (extension)	*0.39%*	*0.21%*	*0.41%*	*0.47%*	*5*



1. Model Hierarchy: LSTMs Win
   a. Top 3 match the paper: CNN-LSTM (0.35%), LSTM-MLP (0.33%), LSTM (0.30%).
   b. LSTMs significantly outperform simpler models: MLP (0.17%) and ARX (0.27%).
2. Input Features: OF beats LOB
   a. Raw LOB is a disaster for neural nets: MLP drops to −2.38% on LOB vs. +0.17% on OF.
   b. OF is better for 5 of 6 stocks (the exception is small-tick UBER).
3. Horizon Decay: Alpha is Short-Lived
4. Stock Characteristics: Tick Size Matters
   a. Rank correlation with tick size is +0.94. Large-tick stocks (BAC, NKE) reach 0.6% – 0.8%.
   b. Small-tick (UBER) remains noisy and near zero.

Order Flow (OF) vs. Order Flow Imbalance (OFI)

- The Definitions:
  - OF (20 features): Keeps bid and ask volume changes separate. Allows asymmetric learning.
  - OFI (10 features): bOF − aOF. A compressed measure of net buy/sell pressure.
- What the Original Paper Found (Appendix B.1):
  - LSTM-based models: Performance is qualitatively similar on OF and OFI.
  - ARX & MLP models: OFI significantly improves performance.
  - *ARX:* R^2 jumps from 0.1% to 0.5%.
  - *MLP:* R^2 improves from 0.25% to 0.3%.
  - *Reason:* Fewer input features = fewer parameters = less overfitting.
- Our Replication on IEX DEEP:
  - We observed the exact same pattern: LSTM-MLP performed equally well on both, while the simpler MLP benefited from the noise reduction of OFI.

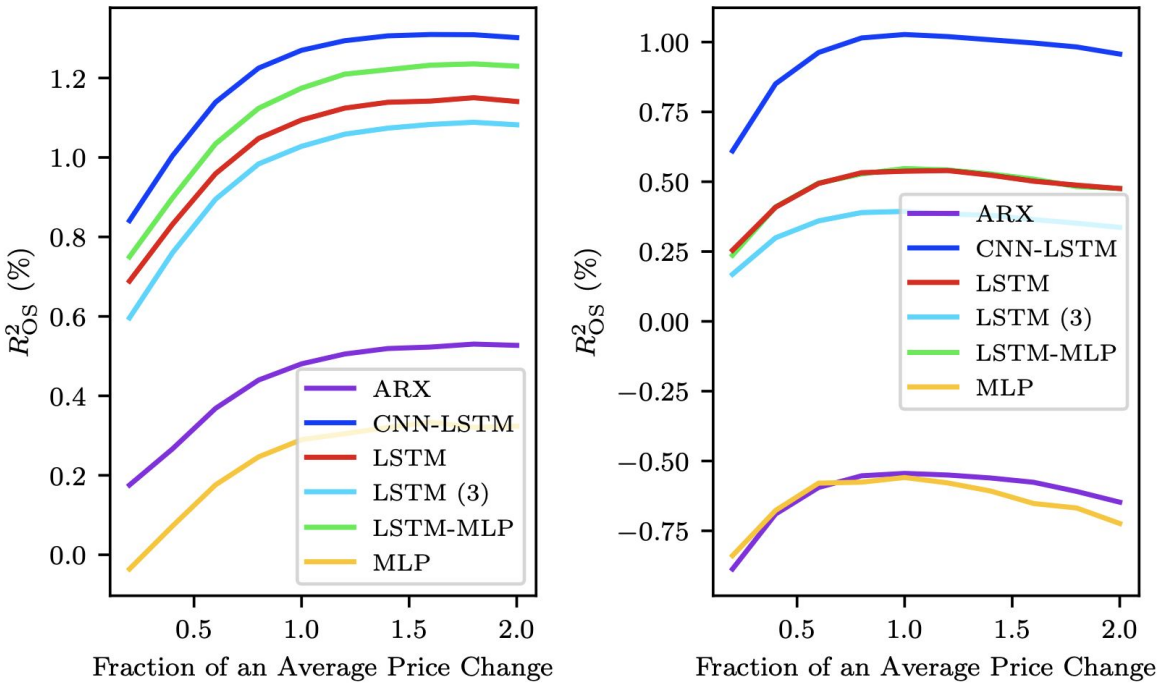
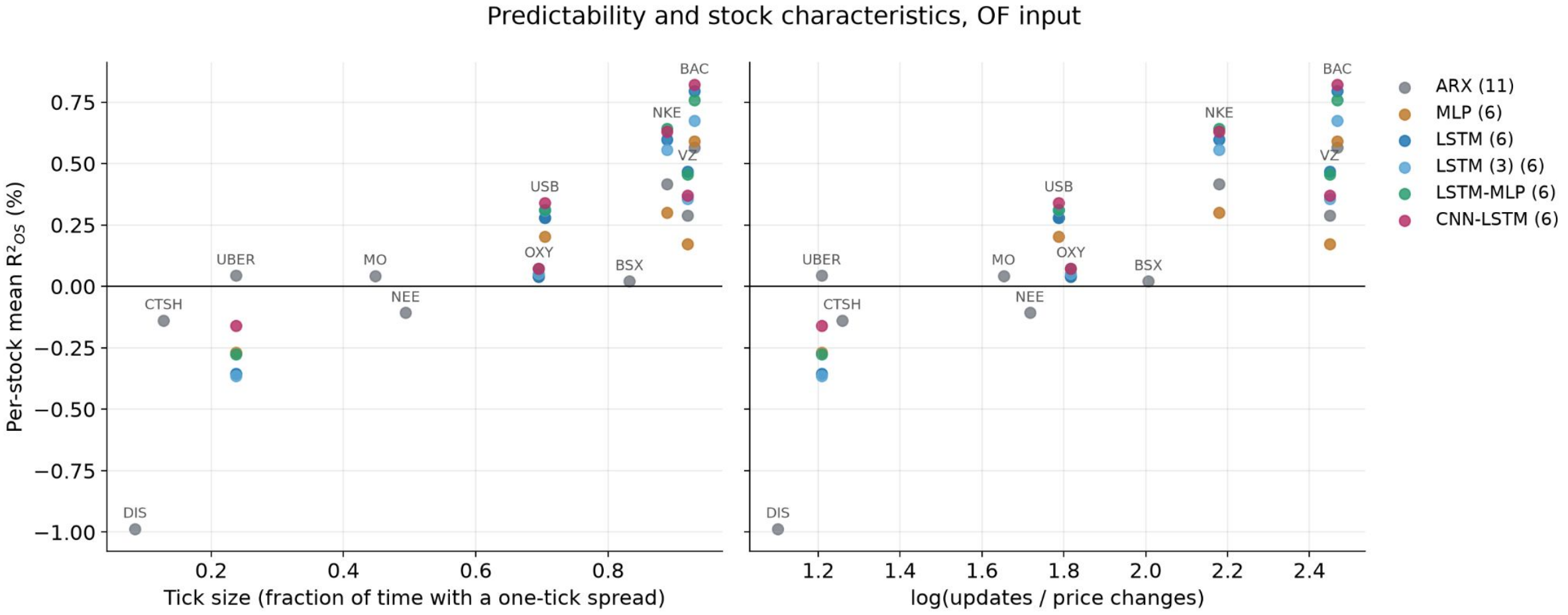


FIGURE 9. Results from robustness checks of the out-of-sample forecasting performance of models at different horizons from 10 levels of standard OFIs (left panel) and order book states with bid and ask prices removed (right panel). Model performance is measured as the average R^2_{OS} across days and stocks. Horizons are given as the fraction of an average price change for each stock. Each model is trained using a rolling-window out-of-sample methodology across 11 months of data.


Predictability and stock characteristics, OF input
Per-stock mean R^2_{OS} (%)
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
BAC
NKE
VZ
USB
OXY
BSX
UBER
MO
NEE
CTSH
DIS
0.2
0.4
0.6
0.8
Tick size (fraction of time with a one-tick spread)
1.2
1.4
1.6
1.8
2.0
2.2
2.4
log(updates / price changes)
ARX (11)
MLP (6)
LSTM (6)
LSTM (3) (6)
LSTM-MLP (6)
CNN-LSTM (6)

Extensions Beyond the Paper

- **Ensemble:** test whether combining linear ARX with CNN-LSTM improves forecasts, with no extra model training.
- **Pooled ARX:** test whether one model can learn patterns shared across stocks and replace separate models.
- **Ridge vs Adam:** check whether the estimation method changes accuracy while the ARX architecture stays the same.

Experiment	Implementation	Result: mean R^2_{OS}
ARX + CNN-LSTM ensemble	50/50 average of ARX and CNN-LSTM forecasts.	Better: 0.39% vs 0.35% for CNN-LSTM and 0.27% for ARX (OF, 6 stocks).
Pooled ARX	One ridge ARX for all 11 stocks in each rolling window.	Worse: 0.03% vs 0.07% for separate ARX models (OFI, 11 stocks).
ARX: ridge vs Adam	Same linear architecture, two estimation methods.	Ridge is better: OF 0.27% vs −0.25%, OFI 0.28% vs 0.05% (6 stocks).

Conclusion

- Stationarity is the key. Order Flow (OF) and Order Flow Imbalance (OFI) beat raw order book states - every time, for every model.
- Memory matters. Architectures with an LSTM layer consistently outperform linear models and plain MLPs.
- Alpha is short-lived. Predictive power peaks around two average price changes and then decays.
- Microstructure decides. "Information-rich" large-tick stocks are far more predictable than small-tick ones.
- Our contribution: We reproduced all of the above on IEX DEEP, and showed it is feasible on consumer hardware.
- Conducted experiments and showed good results (for example ARX + CNN-LSTM ensemble)

Repository Structure

SOURCE PACKAGE

```
src/deepofi/
├── cli.py                          train / evaluate / predict
├── engine.py                       training / load weights
├── config.py                       read YAML settings
├── iex/
│   └── deep.py, hist.py            DEEP parser / HIST archive
├── data_process/
│   ├── book.py                     rebuild order books
│   ├── clean.py                    session / book filters
│   ├── characteristics.py          daily stock statistics
│   ├── universe.py                 stock selection
│   ├── features.py                 OF / OFI / LOB inputs
│   ├── targets.py                  future mid-price changes
│   ├── preprocessing.py            winsorization / z-score
│   ├── windows.py                  sequences / data loaders
│   ├── dataset.py                  books to model inputs
│   ├── splits.py                   rolling date splits
│   └── synthetic.py                artificial data
├── models/
│   ├── __init__.py                 model registry
│   ├── arx.py, mlp.py              linear / dense networks
│   ├── lstm.py, lstm3.py           1 / 3 LSTM layers
│   └── lstm_mlp.py, cnn_lstm.py    hybrid networks
└── evaluation/
    └── metrics.py, comparison.py   scoring / comparisons
```

REPOSITORY ROOT

```
├── README.md                                  overview / results / use
├── DEVIATIONS.md                              changes from the paper
├── pyproject.toml                             package / dependencies
├── requirements.txt                           editable installation
├── .gitignore                                 exclude local files
├── configs/
│   └── pipeline.yaml, tickers.yaml            settings / universe
├── data/
│   └── README.md, samples/*.csv               formats / examples
├── scripts/
│   ├── download_days.py, overnight.py download
│   ├── pilot_day.py                           stock screening
│   ├── build_dataset.py                       books / metadata
│   ├── freeze_dataset.py, check_dataset.py integrity
│   ├── make_samples.py                        example CSV files
│   ├── train_synthetic.py, train_one_day.py smoke tests
│   ├── lr_check.py                            learning-rate check
│   ├── extensions.py                          ensemble / pooled ARX
│   └── make_report.py                         final figures / tables
├── weights/<model>/<input>/.../*.pt           saved parameters
├── results/
│   ├── <model>/<input>/.../                   metrics / history CSV
│   ├── model_comparison/                      comparison PNG / CSV
│   └── report/                                final figures / tables
└── tests/test_*.py                            pipeline / model tests
```


HSE UNIVERSITY
EST 1992